

# Guia de integração com pytest

Configuração segura da ponte local para executar testes Python e vincular automaticamente resultados, cobertura e defeitos.

- |   |                                                |
|---|------------------------------------------------|
| 1 | Marque cada teste com o ID CT-xxx              |
| 2 | Execute pytest no ambiente correto             |
| 3 | Importe cobertura e resultados automaticamente |
| 4 | Diagnostique testes sem vínculo e falhas       |

# Sumario

Siga as etapas na ordem apresentada. Os exemplos consideram Windows e uma ponte local em 127.0.0.1:8765.

|                                                         |          |
|---------------------------------------------------------|----------|
| <b>O que a integração faz</b>                           | <b>3</b> |
| <b>1. Identificar os casos nos testes</b>               | <b>3</b> |
| <b>2. Instalar as dependências</b>                      | <b>3</b> |
| <b>3. Configurar a aplicação</b>                        | <b>4</b> |
| <b>4. Executar o teste de mutação</b>                   | <b>4</b> |
| Cosmic Ray no Windows nativo . . . . .                  | 5        |
| mutmut 3 em Linux ou WSL . . . . .                      | 5        |
| <b>5. Como os dados são vinculados</b>                  | <b>5</b> |
| <b>6. Segurança</b>                                     | <b>5</b> |
| <b>7. Solução de problemas</b>                          | <b>6</b> |
| Ponte indisponível . . . . .                            | 6        |
| pytest não está instalado . . . . .                     | 6        |
| Cobertura não coletada . . . . .                        | 6        |
| Teste não vinculado . . . . .                           | 6        |
| Testes falharam, mas a sincronização terminou . . . . . | 6        |
| mutmut informa que requer WSL . . . . .                 | 6        |
| Configuração do Cosmic Ray não encontrada . . . . .     | 6        |

# O que a integração faz

O V&V TestLab usa uma ponte local para executar o repositório Python e transportar os resultados para o navegador. Em uma única sincronização ela:

- 1 Executa `pytest` no repositório autorizado.
- 2 Carrega o plugin `vvtestlab_pytest_plugin`.
- 3 Produz `.vvtestlab/junit.xml`.
- 4 Produz `.vvtestlab/coverage.json` quando `pytest-cov` está instalado.
- 5 Consolida testes parametrizados ou múltiplos testes sob o mesmo `CT-xxx`.
- 6 Envia execuções, cobertura e testes sem vínculo para a aplicação.
- 7 Atualiza automaticamente a matriz de rastreabilidade.
- 8 Abre um defeito quando o caso falha, se essa opção estiver habilitada.
- 9 Executa `mutmut` ou `cosmic-ray` e registra automaticamente o escore de mutação.

O navegador não consegue ler pastas nem iniciar processos por conta própria. A ponte existe para realizar essas operações localmente sem criar um servidor remoto ou enviar o código-fonte para terceiros.

## 1. Identificar os casos nos testes

Forma recomendada:

```
import pytest

@pytest.mark.vv_case("CT-001")
def test_idade_minima_valida():
    resultado = cadastrar_usuario(nome="Ana", idade=18)
    assert resultado.sucesso
```

O mesmo caso pode ter testes parametrizados:

```
import pytest

@pytest.mark.vv_case("CT-002")
@pytest.mark.parametrize("idade", [17, 121])
def test_idade_fora_dos_limites(idade):
    assert not cadastrar_usuario(nome="Ana", idade=idade).sucesso
```

O resultado consolidado será **Falhou** se qualquer parametrização falhar, **Bloqueado** se houver apenas testes ignorados/bloqueados e **Aprovado** quando todos passarem.

Também é possível colocar o identificador no nome:

```
def test_CT_003_login_com_senha_correta():
    ...
```

O marcador é preferível porque o nome do teste pode ser refatorado sem perder o vínculo.

## 2. Instalar as dependências

Ative o ambiente virtual do repositório e instale:

```
python -m pip install pytest pytest-cov
# Em um ambiente Python separado, se necessário:
python -m pip install cosmic-ray
```

Se o projeto usa `.venv`, execute a ponte com esse mesmo Python. No Windows:

```
& "C:\caminho\repositorio\.venv\Scripts\python.exe" integration\vv_bridge.py `
--repo "C:\caminho\repositorio" `
--cov-source nome_do_pacote `
--mutation-tool cosmic-ray `
--mutation-python .venv-mutation/Scripts/python.exe `
--cosmic-ray-config cosmic-ray.toml `
--cosmic-ray-selector scripts/selecionar_mutantes.py `
--pytest-args tests
```

O comando acima deve ser executado na pasta do V&V TestLab. `--cov-source` pode ser o pacote principal, `src` ou `..`. Os argumentos depois de `--pytest-args` são encaminhados ao pytest sem usar shell.

### 3. Configurar a aplicação

Em **Configurar**, preencha:

- **Caminho local do repositório:** caminho usado para exibir o comando correto.
- **URL da ponte:** normalmente `http://127.0.0.1:8765`.
- **Criar defeito ao falhar:** "Sim" para abrir automaticamente um defeito rastreado.
- **Ferramenta de mutação:** `auto`, `mutmut` ou `cosmic-ray`.
- **Configuração do Cosmic Ray:** caminho relativo do TOML dentro do repositório.
- **Python da mutação:** caminho relativo de outro ambiente, quando Cosmic Ray exige dependências incompatíveis com as do aplicativo.
- **Seletor de mutantes:** script relativo opcional para definir uma amostra reproduzível.
- **Etapa atual:** determina em qual etapa a cobertura importada será registrada.

Depois acesse **Integração Python**:

- 1 Clique em **Verificar ponte**.
- 2 Confira se pytest e cobertura aparecem como disponíveis.
- 3 Clique em **Executar e sincronizar**.
- 4 Corrija os testes listados como "não vinculados".

### 4. Executar o teste de mutação

Faça esta etapa somente depois que a suíte normal estiver passando. Em **Integração Python**, clique em **Executar mutação**. A ponte cria um registro `MUT-nnn` e uma métrica da etapa **Baseado em defeitos** com total, mortos, sobreviventes e escore.

## Cosmic Ray no Windows nativo

Para o hotel, a ponte executa o pytest com Python 3.9 e o Cosmic Ray com Python 3.12, indicado em `--mutation-python`. O seletor `--cosmic-ray-selector scripts/selecionar_mutantes.py` cria a sessão com **todos os 189 mutantes** das funções dos três requisitos, sem amostragem. A ponte executa essa sessão e lê `cosmic-ray dump`, registrando os resultados atuais no painel. Use `output/hotel-management-testado/iniciar-integracao.ps1` (código corrigido) ou `iniciar-integracao.ps1 -Versao original` (código original, para ver os defeitos surgirem como falhas) e importe `output/hotel-vvtestlab-projeto-inicial.json`, que começa sem execuções e métricas.

O modo `auto` prioriza o Cosmic Ray no Windows. Crie `cosmic-ray.toml` na raiz do repositório testado, por exemplo:

```
[cosmic-ray]
module-path = "src/meu_pacote"
timeout = 30.0
excluded-modules = []
test-command = "python -m pytest -q"

[cosmic-ray.distributor]
name = "local"
```

A ponte cria uma sessão nova em `.vvtestlab/cosmic-ray-aaaamdd-hhmmss.sqlite`, executa os mutantes e importa o dump JSON. Quando há seletor, ele prepara a sessão e grava um manifesto ao lado; sem seletor, a ponte usa `cosmic-ray init`.

## mutmut 3 em Linux ou WSL

Configure `--mutation-tool mutmut`. A ponte executa `mutmut run`, gera o relatório oficial com `mutmut export-cicd-stats` e importa `mutants/mutmut-cicd-stats.json`. O mutmut 3+ exige suporte a `fork`; no Windows, execute a ponte dentro do WSL.

## 5. Como os dados são vinculados

```
@pytest.mark.vv_case("CT-002")
|
v
JUnit XML: vv_case_id=CT-002
|
v
EXE-nnn -> CT-002 -> CE-nnn -> REQ-nnn
|
+-- cobertura -> MET-nnn
`-- falha -> DEF-nnn
```

Cada execução automática armazena `source=pytest` e o identificador `PYTEST-aaaamdd-hhmmss`. O histórico `SYNC-nnn` registra duração, exit code, quantidade mapeada, quantidade sem vínculo e cobertura.

## 6. Segurança

- A ponte aceita conexões somente em `127.0.0.1`.
- O repositório é definido na inicialização; a interface não pode enviar outro caminho.
- O processo usa uma lista de argumentos e `shell=False`.
- O código e os resultados não são enviados para serviços externos.
- pytest tem limite de 30 minutos; mutação tem limite de 2 horas.

- Encerre a ponte com `Ctrl+C` quando terminar.

Adicione ao `.gitignore` do repositório testado:

```
.vvtestlab/  
mutants/
```

## 7. Solução de problemas

### Ponte indisponível

Confirme que o terminal continua aberto e que a URL/porta da aplicação é a mesma mostrada pela ponte.

### pytest não está instalado

A ponte precisa ser iniciada com o Python do ambiente que contém pytest. Verifique com:

```
python -m pytest --version
```

### Cobertura não coletada

Instale `pytest-cov` no mesmo ambiente e reinicie a ponte.

### Teste não vinculado

Confira se o ID existe no V&V TestLab e se o marcador usa o formato `CT-001`. IDs detectados que não existem no sistema são mantidos na lista de não vinculados.

### Testes falharam, mas a sincronização terminou

Isso é esperado: pytest retorna exit code 1 quando existem falhas. O V&V TestLab ainda importa o JUnit, registra as execuções como **Falhou** e cria os defeitos configurados.

### mutmut informa que requer WSL

O mutmut 3+ não executa no Windows nativo. Inicie a ponte no WSL ou selecione `cosmic-ray` em **Configurar** e reinicie a ponte com o comando atualizado.

### Configuração do Cosmic Ray não encontrada

Crie o TOML dentro do repositório autorizado e confirme o caminho relativo em **Configurar**. A ponte rejeita caminhos externos por segurança.